VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRẦN TRUNG HIẾU – 52400047
NGUYỄN ĐỨC NHẬT HUY – 52400122

CONTAINERS IN WEB DEVELOPMENT AND DEPLOYMENT (DOCKER)

MIDTERM ESSAY

WEB PROGRAMMING & APPLICATIONS

HO CHI MINH CITY, 2026

VIETNAM GENERAL CONFEDERATION OF LABOR
TON DUC THANG UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY



TRẦN TRUNG HIẾU – 52400047
NGUYỄN ĐỨC NHẬT HUY – 52400122

CONTAINERS IN WEB DEVELOPMENT AND DEPLOYMENT (DOCKER)

MIDTERM ESSAY

WEB PROGRAMMING & APPLICATIONS

Advised by

MSc. Nguyễn Thái Duy

HO CHI MINH CITY, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Mr. Nguyễn Thái Duy, our instructor, for his valuable guidance and support throughout this midterm essay on containers in web development and deployment. His lectures gave us the foundation in web technology on which this work is built. We also thank the Faculty of Information Technology of Ton Duc Thang University for the learning environment and resources that made the project possible. Despite our efforts, the report may still contain shortcomings, and we look forward to receiving comments that will help us improve.

Ho Chi Minh city, ... 2026

Authors

(Signature and full name)

Trần Trung Hiếu

Nguyễn Đức Nhật Huy

DECLARATION OF AUTHORSHIP

We hereby declare that this is our own project and is guided by Mr. Nguyễn Thái Duy. The content, research and results contained herein are honest and have not been published in any form before. The data in the tables for analysis, comments and evaluation are collected by the authors from different sources, which are clearly stated in the reference section.

In addition, the project also uses some comments, assessments as well as data of other authors and other organizations, with citations and annotated sources.

If something wrong happens, we will take full responsibility for the content of our project. Ton Duc Thang University is not related to the infringing rights or the copyrights that we give during the implementation process (if any).

Ho Chi Minh city, ... 2026

Authors

(Signature and full name)

Trần Trung Hiếu

Nguyễn Đức Nhật Huy

CONTAINERS IN WEB DEVELOPMENT AND DEPLOYMENT (DOCKER)

ABSTRACT

This report surveys container technology and its role in the development and deployment of web applications. The theoretical part covers the history of containers, the Linux kernel mechanisms behind them, the Docker architecture, images and layers, networking, storage, orchestration with Kubernetes and security practices, and it compares containers with virtual machines and alternative tools. To test the theory, we built DockMart, a multi-container online store written in Node.js and React, which runs unchanged on Docker Compose and on a three-node Kubernetes cluster. Our experiments measured image size, scalability and availability under failure. A multi-stage Alpine image was 6.0 times smaller than a naive image, throughput grew 4.8 times from one to six replicas, and rolling updates, rollbacks and a node drain completed with no failed requests at 100 requests per second. The results confirm that containers deliver portability, consistency and efficiency, on the condition that the application is designed to be stateless and to handle failure.

TABLE OF CONTENTS

LIST OF FIGURES ..... **vii**

LIST OF TABLES ..... **ix**

ABBREVIATIONS ..... **x**

CHAPTER 1. INTRODUCTION **1**

1.1 Reasons for choosing the topic 1

1.2 Objectives 1

1.3 Scope and approach 2

1.4 Structure of the report 2

CHAPTER 2. THEORETICAL BASIS **3**

2.1 A short history of containers 3

2.2 Linux foundations 4

2.2.1 Namespaces *4*

2.2.2 Control groups *4*

2.2.3 Union filesystems and capabilities *5*

2.3 Docker architecture 5

2.4 Images, layers and the build cache 6

2.5 Containers at run time 7

2.6 Networking and storage 8

2.7 Multi-container applications with Docker Compose 9

2.8 Orchestration and scalability 9

2.9 Container security 11

2.10 Advantages and limitations of containers 11

2.11 Comparative analysis 12

2.11.1 Containers and virtual machines *12*

2.11.2 Docker and other container engines *14*

2.11.3 Docker Compose, Swarm, Kubernetes and Nomad *15*

2.11.4 Containers and platform services *16*

CHAPTER 3. DESIGNING AND IMPLEMENTING 17

3.1 Requirement analysis 17

3.1.1 Purpose and target audience *17*

3.1.2 Functional requirements *17*

3.1.3 Non-functional requirements *17*

3.2 System design 18

3.2.1 System architecture *18*

3.2.2 Data design *19*

3.2.3 Order processing flow *20*

3.2.4 Technology choices *21*

3.3 Implementation 22

3.3.1 Image design *22*

3.3.2 The Compose stack *23*

3.3.3 Dynamic load balancing *24*

3.3.4 Stateless replicas *24*

3.3.5 Reliability *25*

3.3.6 Security hardening *26*

3.3.7 Monitoring *27*

3.3.8 Kubernetes deployment *27*

3.3.9 Development workflow and delivery *28*

3.4 Website walkthrough 29

CHAPTER 4. RESULT 32

4.1 Test environment and method 32

4.2 Functional testing 32

4.3 Image build strategies 33

4.4 Start-up time and resource usage 35

4.5 Horizontal scaling 36

4.6 Failure behaviour on Docker Compose 37

4.7 Kubernetes 39

4.7.1 Availability *39*

4.7.2 Autoscaling *40*

4.7.3 Network policies *41*

4.8 Problems encountered 41

4.9 Discussion 43

CHAPTER 5. CONCLUSION 45

5.1 Conclusion 45

5.2 Potential and Development Direction 45

REFERENCES 47

APPENDIX A. RUNNING THE PROJECT 50

APPENDIX B. REST API 51

LIST OF FIGURES

Figure 2.1: Milestones in the history of containers.....3

Figure 2.2: Namespaces isolate what a container sees, and cgroups limit what it uses 5

Figure 2.3: Components involved in running a container with Docker.....6

Figure 2.4: An image is a stack of shared read-only layers, and each container adds one writable layer.....7

Figure 2.5: Container lifecycle and the path taken by a restart policy 8

Figure 2.6: Kubernetes architecture. The control plane decides, and the nodes run containers 10

Figure 2.7: A rolling update replaces Pods one at a time, keeping capacity constant 10

Figure 2.8: Virtual machines carry a full guest operating system, while containers share the host kernel.....13

Figure 3.1: DockMart system architecture on Docker Compose.....19

Figure 3.2: Entity-relationship diagram of the DockMart database 20

Figure 3.3: Sequence of an order, from checkout to the live status update 21

Figure 3.4: Multi-stage build of the backend image. Only the production stage ships 22

Figure 3.5: Start-up order, driven by health checks instead of fixed delays.....23

Figure 3.6: How the Ops Center discovers replicas without access to the Docker socket.....25

Figure 3.7: Delivery pipeline defined in the GitHub Actions workflow 28

Figure 3.8: Storefront, with the cache status and the replica that served the page ...29

Figure 3.9: Live order tracking, updated through Server-Sent Events 30

Figure 3.10: Ops Center just after one replica crashed: its uptime was reset, its restart count rose, and the event log recorded the crash 31

Figure 3.11: Grafana dashboard, with replicas discovered automatically by Prometheus 31

Figure 4.1: All services running and healthy on Docker Compose (real output) 33

Figure 4.2: Size of the same application under four build strategies 34

Figure 4.3: Build time, first build and rebuild after a one-line change 35

Figure 4.4: Memory used by each container at idle 36

Figure 4.5: Throughput and latency as replicas are added 36

Figure 4.6: Ready Pods by version during a rolling update 39

Figure 4.7: DockMart Pods spread across the two worker nodes of the cluster (real output) 40

Figure 4.8: Autoscaling under load: replicas and average CPU use 41

LIST OF TABLES

Table 2.1: Advantages of containers and their underlying mechanisms 12

Table 2.2: Comparison of containers and virtual machines.....13

Table 2.3: Comparison of container engines 14

Table 2.4: Comparison of orchestration options.....15

Table 3.1: Functional requirements.....17

Table 3.2: Non-functional requirements 18

Table 3.3: Technology stack 21

Table 3.4: Where each kind of state lives, so that replicas stay interchangeable 25

Table 3.5: Security measures applied to the application containers 26

Table 3.6: How Compose concepts map to Kubernetes objects.....27

Table 4.1: Test environment 32

Table 4.2: Automated test results.....33

Table 4.3: Comparison of image build strategies for the same backend 34

Table 4.4: Throughput and latency by number of API replicas.....37

Table 4.5: Failure experiments on Docker Compose.....37

Table 4.6: Measured connectivity between services.....39

Table 4.7: Availability experiments on Kubernetes.....40

Table 4.8: Problems met during the project, and how we solved them 42

Table 4.9: Non-functional requirements and results.....43

Table 4.10: What each platform delivered in our measurements 43

ABBREVIATIONS

API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Delivery
CLI	Command-Line Interface
CNCF	Cloud Native Computing Foundation
CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
DNS	Domain Name System
HPA	HorizontalPodAutoscaler
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LXC	Linux Containers
OCI	Open Container Initiative
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SSE	Server-Sent Events
TTL	Time To Live
VM	Virtual Machine
WSL	Windows Subsystem for Linux
YAML	YAML Ain't Markup Language

CHAPTER 1. INTRODUCTION

1.1 Reasons for choosing the topic

A web application rarely fails because of its business logic alone. It fails when the database driver on the server differs from the one on a developer's laptop, when a release needs a maintenance window, or when traffic doubles and nobody can add capacity quickly. The phrase "it works on my machine" has survived for decades because the environment around the code has traditionally been configured by hand, and it drifts.

Containers address this problem by packaging an application together with everything it needs to run, in a format that behaves the same on any host. The approach has moved well beyond early adopters. In the 2025 Stack Overflow Developer Survey, 71.1% of respondents reported using Docker, which placed it among the most widely used tools in the survey (Stack Overflow, 2025). The 2025 annual survey of the Cloud Native Computing Foundation reported that 82% of container users run Kubernetes in production, up from 66% in 2023 (Cloud Native Computing Foundation, 2026). For a student of web programming, deployment is therefore no longer a separate speciality. It is part of how web applications are built.

We chose this topic for a second reason: it can be demonstrated, not only described. Claims such as "containers are portable" or "orchestration keeps services available" can be tested by running a real system, breaking it on purpose and measuring what happens. That is the approach this report takes.

1.2 Objectives

The essay has three objectives. The first is to explain how containers work, from the Linux kernel features underneath them to the orchestrators that run them at scale. The second is to compare containers with the alternatives, so that the reader can decide when they are the right tool and when they are not. The third is to apply these ideas in a working website, and to back each claim about portability, consistency, efficiency and scalability with measurements taken on that website.

1.3 Scope and approach

The demonstration is DockMart, an online store that we built with Node.js and React and split into containers. It covers the two deployment targets suggested by the assignment: Docker Compose on a single host, and Kubernetes on a three-node cluster. The same images run on both. We did not build features for their own sake. Each one exists because it exposes a container concept: a background worker shows that one image can serve two roles, a live dashboard makes load balancing visible, and a chaos button makes self-healing observable.

The report does not cover running a production cluster on a public cloud, service meshes, or multi-region deployment. HTTPS termination is also outside the demonstration, because it belongs to the deployment topic rather than to containers. Chapter 5 returns to these points as future work.

1.4 Structure of the report

Chapter 2 surveys the theory, from the history of containers and the kernel mechanisms to Docker, orchestration and security, and ends with a comparison of containers with virtual machines and other tools. Chapter 3 analyses the requirements of the demonstration, presents its design and explains how it was implemented. Chapter 4 reports our experiments and the problems we met. Chapter 5 concludes the report and suggests further work.

CHAPTER 2. THEORETICAL BASIS

2.1 A short history of containers

The idea of isolating a process is older than the web. In 1979, Version 7 Unix introduced chroot, which changes the directory that a process sees as the root of the filesystem. FreeBSD Jails extended this idea in 2000 by also confining users and networking (Kamp & Watson, 2000), and Solaris Zones followed in 2004 (Price & Tucker, 2004). These systems proved that one kernel could safely host several isolated environments, with far less overhead than a hypervisor (Soltesz et al., 2007).

Linux gained the same abilities gradually. Control groups were merged into the kernel in 2008, and LXC combined them with namespaces into a usable container system (Bernstein, 2014). What LXC did not provide was a convenient way to package an application and move it between machines. Docker supplied that missing piece. Solomon Hykes presented it at PyCon in March 2013 (Hykes, 2013), and its contribution was less a new isolation technique than a developer experience: a simple text file that describes an image, a layered image format, and a public registry for sharing images (Merkel, 2014).

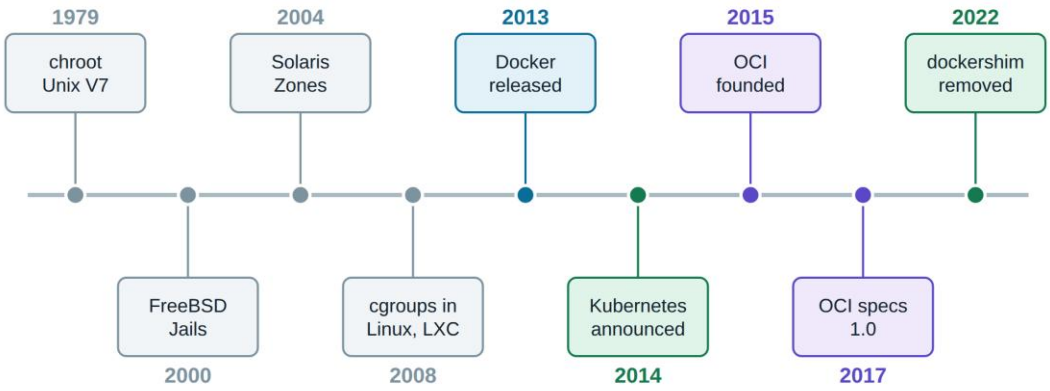


Figure 2.1: Milestones in the history of containers

Google had been running its internal workloads in containers for years under a cluster manager called Borg (Verma et al., 2015). In 2014 it released Kubernetes, an open-source system built on the lessons of Borg and its successor Omega (Burns

et al., 2016). In 2015 Docker, CoreOS and others founded the Open Container Initiative (OCI), which standardised the image format and the runtime so that no single vendor owns them (Open Container Initiative, n.d.). The value of this standard became clear in 2022, when Kubernetes removed its special integration with Docker Engine. Images built with Docker kept working unchanged, because they are OCI images (Kubernetes, 2022).

2.2 Linux foundations

A container is not a lightweight virtual machine. It is an ordinary Linux process that the kernel treats specially. Three kernel features create the illusion of a separate machine, and Figure 2.2 shows how they combine.

2.2.1 Namespaces

A namespace gives a process its own view of a global resource (Kerrisk, n.d.-b). In a PID namespace, the first process believes that it is PID 1 and cannot see processes outside. A network namespace provides separate interfaces, routing tables and ports, which is why two containers can both listen on port 8080. Mount, UTS, IPC and user namespaces do the same for the filesystem tree, the hostname, shared memory and user identifiers.

2.2.2 Control groups

Namespaces limit what a process can see. Control groups limit what it can use (Kerrisk, n.d.-c). A cgroup caps CPU time, memory and other resources for a group of processes. Our API containers are limited to half a CPU core and 256 MB of memory, and the kernel enforces both limits. If a replica leaks memory, the kernel stops that replica rather than the whole server. This is what makes it safe to run many services on one host.

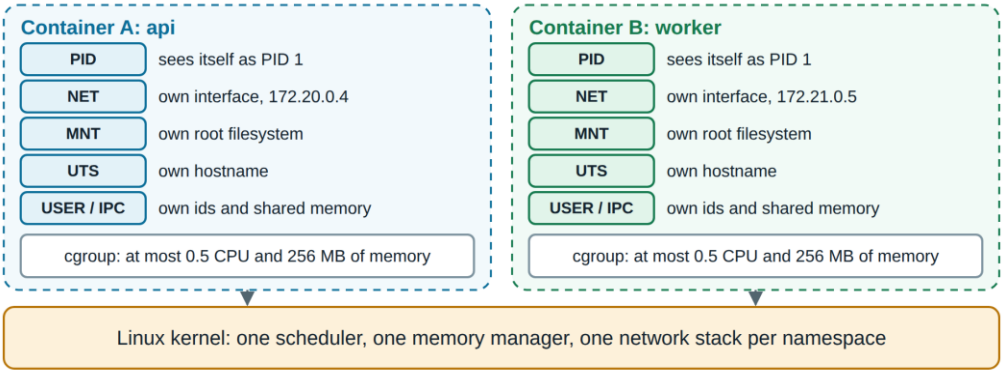


Figure 2.2: Namespaces isolate what a container sees, and cgroups limit what it uses

2.2.3 Union filesystems and capabilities

A union filesystem such as OverlayFS presents several directories as one (Docker Inc., n.d.-h). This allows images to be stored as stacked layers, as Section 2.4 explains. Linux capabilities split the power of the root user into separate privileges, such as binding a low port or changing file ownership (Kerrisk, n.d.-a). A process can drop the privileges it does not need, which we use in Chapter 3 to run containers with no capabilities at all.

2.3 Docker architecture

Docker follows a client-server design (Docker Inc., n.d.-a). The docker command is a client that calls a REST API on the daemon, dockerd. The daemon handles images, networks and volumes, and delegates running containers to containerd, which in turn uses runc to create each container through the kernel features above. After starting a container, runc exits and a small shim process stays behind to hold the container's standard streams and exit status. As a result, containers do not depend on containerd staying up, and with the live restore option they also survive a restart of the Docker daemon.

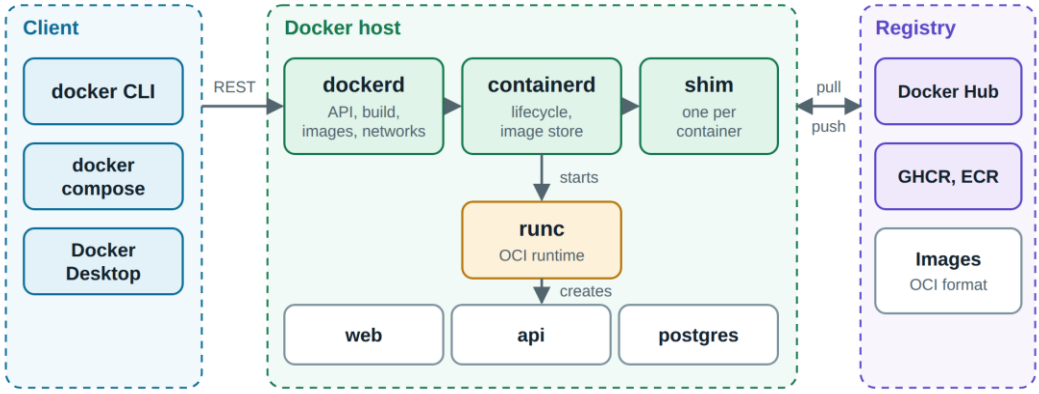


Figure 2.3: Components involved in running a container with Docker

Images are stored in registries such as Docker Hub or GitHub Container Registry. Because the format follows the OCI specification, an image built by Docker can be pulled and run by containerd, CRI-O or Podman. Our Kubernetes cluster demonstrates this directly: its nodes run containerd, with no Docker Engine involved, yet they run the images we built with docker build.

2.4 Images, layers and the build cache

An image is a read-only template, built from instructions in a Dockerfile. Each instruction that changes the filesystem creates a layer, and each layer is identified by a hash of its content. When a container starts, Docker adds a thin writable layer on top. Files that the container modifies are first copied up into this layer, a technique known as copy-on-write. Figure 2.4 shows the consequence: three replicas of the same image use one copy of the image on disk, and start almost instantly because nothing needs to be copied.

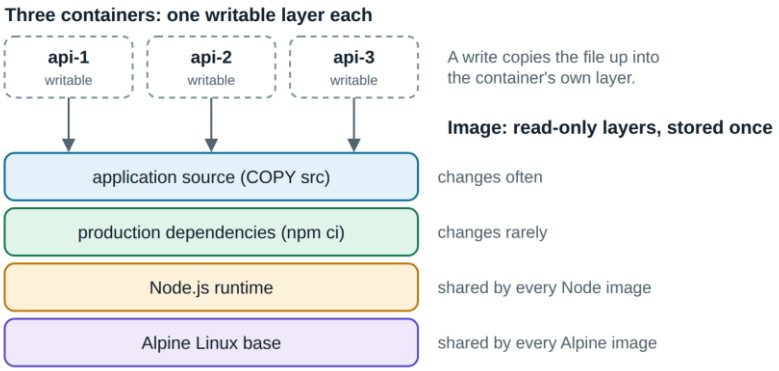


Figure 2.4: An image is a stack of shared read-only layers, and each container adds one writable layer

Layers also explain the build cache. Docker reuses a cached layer until one instruction changes, then rebuilds that layer and everything after it. The order of instructions therefore matters a great deal. Dependencies should be installed before the source code is copied, so that editing one line of code does not trigger a full reinstall (Docker Inc., n.d.-b). We measure the effect of this ordering in Chapter 4.

Multi-stage builds separate what is needed to build software from what is needed to run it. A Dockerfile may contain several FROM instructions, and later stages copy only selected files from earlier ones (Docker Inc., n.d.-c). Compilers, test tools and caches stay behind in stages that are never shipped. A smaller image transfers faster and, more importantly, contains fewer packages that could hold a vulnerability.

2.5 Containers at run time

A container exists only as long as its main process does. It moves through the states in Figure 2.5. Two details of this lifecycle often cause trouble in production, and we designed around both.

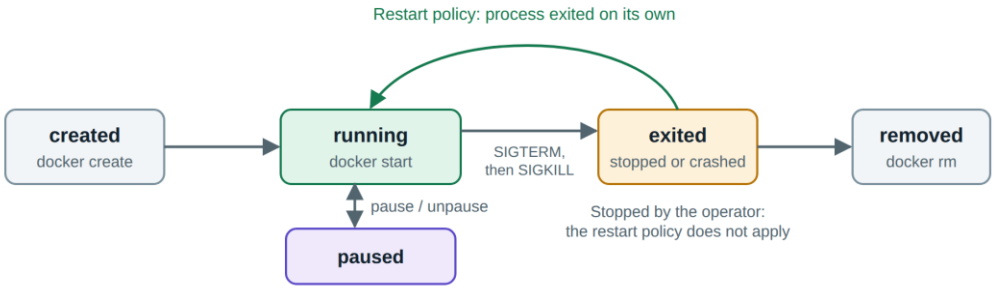


Figure 2.5: Container lifecycle and the path taken by a restart policy

The first detail is signal handling. Stopping a container sends SIGTERM to its main process and, after a grace period of ten seconds by default, SIGKILL. An application can use that interval to finish requests in progress. However, if a Dockerfile starts the application through a shell or through npm, the signal may reach the wrapper and never the application, so every deployment cuts off open requests. The fix is to use the exec form of CMD, so that the application itself runs as the main process.

The second detail is the restart policy, which restarts a container whose process exits. The Docker documentation makes an important exception: a container stopped manually is not restarted until the daemon restarts or the operator starts it again (Docker Inc., n.d.-d). A policy is therefore a recovery mechanism for failures, and it does not override an operator's decision. We learned this during testing, as Section 4.8 describes. Docker can also run a health check inside a container, but on a single host the result is only a status: Docker reports the container as unhealthy and does not replace it. Acting on that signal is the job of an orchestrator.

2.6 Networking and storage

Docker connects containers through virtual networks (Docker Inc., n.d.-e). On a user-defined bridge network, containers reach each other by service name through an embedded DNS server, so no address appears in the configuration. Containers on different networks cannot communicate unless a container joins both. A network marked as internal has no route to the outside world. Only ports that are

explicitly published can be reached from the host. Together, these rules let a project express its security boundaries in configuration.

Because a container's writable layer disappears along with the container, durable data must live elsewhere. Docker offers three options (Docker Inc., n.d.-f). A volume is managed by Docker and survives the container that used it, which makes it the right choice for databases. A bind mount maps a host directory into a container, which is useful in development. A tmpfs mount keeps data in memory and leaves no trace on disk. This separation between replaceable containers and durable data is what allows a container to be deleted without fear.

2.7 Multi-container applications with Docker Compose

Real applications consist of several cooperating processes. Docker Compose describes them in one YAML file: services, their images, networks, volumes, secrets, and the order in which they should start (Docker Inc., n.d.-g). A single command then creates the whole environment, and because the file is plain text, it can be reviewed and versioned along with the code. Compose also supports replicas, resource limits and health-based start-up ordering, which is enough to run a small production system on one host. Its limit is that one host: if the machine fails, everything fails with it, and it has no built-in way to replace containers gradually during an update.

2.8 Orchestration and scalability

An orchestrator manages containers across many machines. Kubernetes is the most widely used, and its central idea is the declarative model. The operator states the desired result, for example three replicas of version 1.1.0, and controllers continuously compare that desired state with reality and correct any difference (Burns et al., 2016). Nobody runs a command to restart a failed container. The system notices the difference and restores the state by itself.

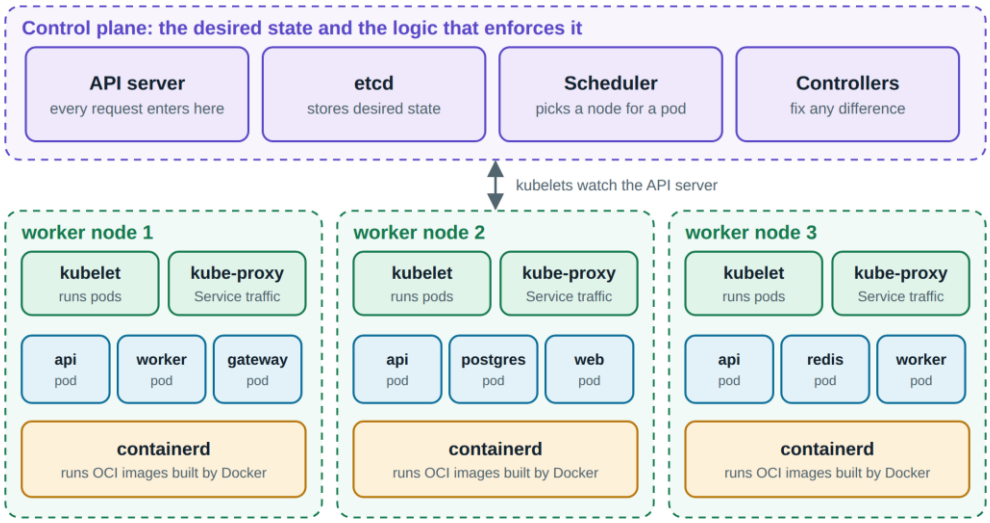


Figure 2.6: Kubernetes architecture. The control plane decides, and the nodes run containers

The control plane holds the desired state in etcd and exposes it through the API server. The scheduler assigns work to nodes, and controllers keep the state converged (Kubernetes, n.d.-a). On each node, the kubelet starts containers through a runtime such as containerd. The basic unit is the Pod, one or more containers that share a network identity. A Deployment manages a set of identical stateless Pods, a StatefulSet gives Pods a stable identity and their own storage, and a Service provides a stable name and address in front of a changing set of Pods.

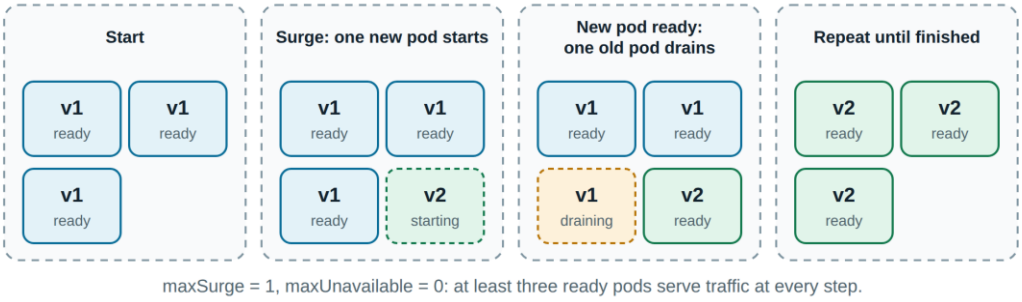


Figure 2.7: A rolling update replaces Pods one at a time, keeping capacity constant

Three features of Kubernetes matter most for scalability and availability. A rolling update replaces Pods gradually, as Figure 2.7 shows, and can be reversed with one command (Kubernetes, n.d.-b). Probes tell the kubelet about a container's health: a failing liveness probe causes a restart, while a failing readiness probe only

removes the Pod from traffic (Kubernetes, n.d.-c). The HorizontalPodAutoscaler changes the number of replicas in response to measured load (Kubernetes, n.d.-d). Scaling out in this way works only if the application is stateless, meaning that any replica can serve any request. This requirement shaped the design of DockMart more than any other, and it follows the twelve-factor methodology for cloud applications (Wiggins, 2017).

2.9 Container security

All containers on a host share one kernel, so a kernel or runtime vulnerability can break isolation. This is not a theoretical concern. CVE-2019-5736 allowed a malicious container to overwrite the host's runc binary and gain root access on the host (National Institute of Standards and Technology, 2019), and CVE-2024-21626 allowed a similar escape through a leaked file descriptor (National Institute of Standards and Technology, 2024). The NIST container security guide therefore recommends treating images, registries, runtimes and hosts as separate layers of defence (Souppaya et al., 2017).

In practice, the guidance from NIST and OWASP (2024) reduces to a short set of habits. Images should be minimal, should be scanned for known vulnerabilities, and should run as a non-root user. Containers should have a read-only filesystem, no unnecessary capabilities and resource limits, so that a compromised process can do little. Secrets should be mounted as files and never stored in an image or an environment variable, because environment variables are visible through docker inspect. Networks should allow only the connections that the application needs. Chapter 3 applies each of these habits to DockMart, and Chapter 4 checks that they took effect.

2.10 Advantages and limitations of containers

Table 2.1 summarises the three advantages named in the assignment, the mechanism behind each, and where the evidence from our own project appears in this report.

Table 2.1: Advantages of containers and their underlying mechanisms

Advantage	Mechanism	Evidence in this report
Portability	OCI images contain the application and its dependencies, and run on any compatible host and runtime	The same images run on Compose and on Kubernetes (Sections 3.3.8 and 4.7)
Consistency	The image is built once and promoted unchanged, and configuration is injected at run time	One Dockerfile produces the development, test and production targets (Section 3.3.1)
Resource efficiency	A shared kernel, shared layers, and no guest operating system	Measured memory use and start-up time (Section 4.4)

Containers also have real limitations. Isolation is weaker than in a virtual machine because the kernel is shared. Windows and Linux containers cannot be mixed on one kernel. Stateful software such as databases needs extra care. Debugging is harder when the image contains no shell, and an orchestrator adds a large amount of operational complexity that a small project may not need. Containers do not fix a poorly designed application either: a service that keeps sessions in memory will still break when it is scaled out. Later chapters return to these limits where we met them.

2.11 Comparative analysis

Choosing a technology means knowing what the alternatives offer. This chapter compares containers with virtual machines, Docker with other container engines, Compose with other orchestrators, and containers with platform services.

2.11.1 Containers and virtual machines

Both technologies run several isolated workloads on one server, but they draw the isolation boundary in different places. A hypervisor virtualises hardware, so each virtual machine boots a complete guest kernel. A container virtualises the operating system, so all containers share the host kernel.

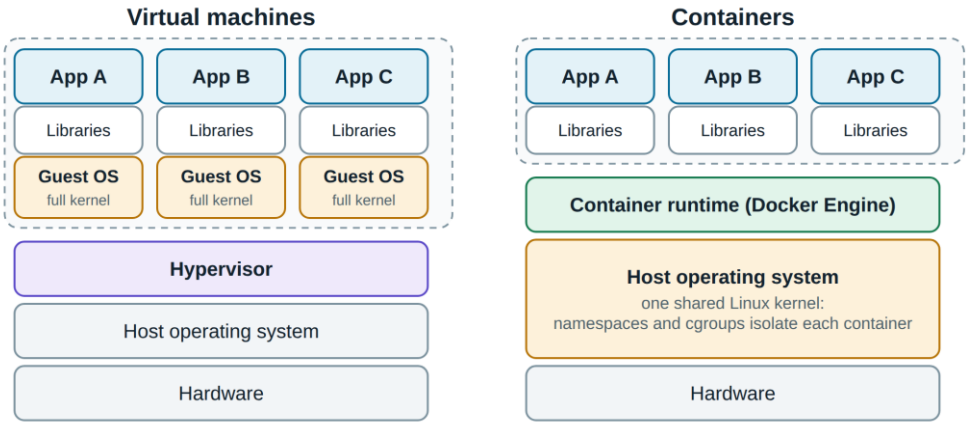


Figure 2.8: Virtual machines carry a full guest operating system, while containers share the host kernel

This single difference explains most of Table 2.2. An early and widely cited IBM study compared Docker with KVM and found that containers matched or exceeded virtual machines in almost every test, while both needed tuning for I/O-intensive work (Felter et al., 2015). The study also noted costs on the container side, in particular the overhead of Docker's network address translation at high packet rates. These details matter: containers have low overhead, but their default settings are not free.

Table 2.2: Comparison of containers and virtual machines

Criterion	Container	Virtual machine
Isolation boundary	Process, on a shared kernel	Hardware, with a separate kernel
Typical start-up	Seconds or less	Tens of seconds to minutes
Disk footprint	Tens to hundreds of megabytes	Gigabytes
Memory overhead	Close to the process itself	A full guest operating system per instance
Security strength	Good when hardened, but one kernel is shared	Stronger: a kernel bug in a guest stays in the guest
Operating systems	Same kernel family as the host	Any operating system
Portability	An image runs on any compatible runtime	Disk images are larger and hypervisor-specific
Best suited to	Many small services, frequent releases	Strong multi-tenant isolation, legacy systems

The two are not rivals in practice. Public clouds usually run containers inside virtual machines, which combines the strong tenant boundary of a hypervisor with the fast packaging of containers. Our test environment is a smaller example of the same layering: Docker Desktop on Windows runs its Linux containers inside a lightweight WSL 2 virtual machine.

2.11.2 Docker and other container engines

Docker is the best-known engine but no longer the only one, and thanks to the OCI standard the choice is not a lock-in decision. Table 2.3 compares the main options. Podman has no central daemon: each container is a child of the user's session, and ordinary users run containers rootless (Podman, n.d.). This removes a privileged daemon from the attack surface. Containerd is the runtime that Docker itself uses, and it is the usual choice on Kubernetes nodes.

Table 2.3: Comparison of container engines

Criterion	Docker Engine	Podman	containerd
Architecture	Central root daemon	No daemon	Daemon for machines, not people
Rootless mode	Optional	Default	Optional
Developer tooling	Excellent: Compose, Desktop, BuildKit	Good, largely compatible	Minimal
Typical use	Development, single hosts	Security-focused hosts	Kubernetes nodes
Licensing note	Engine is open source. Docker Desktop is free for small companies only	Open source	Open source

Licensing deserves a note. The Docker Engine is open source, but Docker Desktop is free only for personal use, education, open-source projects and companies with fewer than 250 employees and less than US$10 million in annual revenue (Docker Inc., n.d.-j). Larger organisations must pay or choose an

alternative. For this project, Docker was the obvious choice: it has the best documentation, the course context allows free use, and the images we produce remain portable to the other engines.

2.11.3 Docker Compose, Swarm, Kubernetes and Nomad

Orchestrators differ mainly in how much they can do and how much they cost to operate. Table 2.4 compares four options, based on their documentation (Docker Inc., n.d.-g; Docker Inc., n.d.-k; HashiCorp, n.d.; Kubernetes, n.d.-a).

Table 2.4: Comparison of orchestration options

Criterion	Compose	Swarm	Kubernetes	Nomad
Number of hosts	One	Many	Many	Many
Learning curve	Low	Low	High	Medium
Self-healing	Restart on exit only	Replaces failed tasks	Replaces and reschedules	Replaces and reschedules
Rolling update	None built in	Yes	Yes, with rollback history	Yes
Autoscaling	Manual	Manual	Built in	Add-on
Network policy	Separate networks	Overlay networks	Per-Pod rules	Through plugins
Ecosystem	Docker	Docker	Very large	HashiCorp
Best suited to	Development, small sites	Small clusters	Large or fast-growing systems	Mixed workloads, not only containers

A common mistake is to treat Kubernetes as the default. It solves problems of scale, and it brings the cost of running a control plane, understanding dozens of object types and securing a large system. A store with a few thousand visitors a day would be well served by Compose on one reliable server. Kubernetes starts to pay off when the system needs to survive the loss of a machine, deploy many times a day, or scale automatically. We implemented both targets so that this difference could be measured, and Chapter 4 shows what each one actually delivers.

2.11.4 Containers and platform services

Containers are not the only way to avoid managing servers. Platform-as-a-Service products and serverless functions hide even more of the infrastructure. They reduce operational work, but they restrict the runtime, can cause cold starts, and tie an application to one vendor. Containers sit between these services and virtual machines: the team keeps control of the runtime and can move between providers, at the price of operating more of the stack itself. Many managed services now accept container images as their deployment format, so a well-built image keeps these later options open.

CHAPTER 3. DESIGNING AND IMPLEMENTING

3.1 Requirement analysis

3.1.1 Purpose and target audience

DockMart has two purposes. To its customers it is an online store for desk equipment, where they can browse products, place orders and follow them in real time. To its real audience, the course instructor and fellow students, it is a laboratory for container concepts. The Ops Center page turns the normally invisible behaviour of a cluster into something that can be watched live during a presentation. The system therefore has three kinds of user: customers, the store administrator, and the operator who runs the containers.

3.1.2 Functional requirements

Table 3.1: Functional requirements

ID	Requirement	Actor
F1	Browse, search, filter and sort products	Customer
F2	Register, sign in and sign out	Customer, admin
F3	Manage a cart and place an order, without overselling stock	Customer
F4	Follow an order's progress live, without reloading the page	Customer
F5	Create, update and delete products, and view all orders	Admin
F6	View all running replicas with version, CPU, memory and restarts	Admin
F7	Send test traffic and see how it is distributed	Admin
F8	Crash a chosen replica, or make it fail its health check	Admin

3.1.3 Non-functional requirements

The non-functional requirements carry the actual subject of the essay. Each one in Table 3.2 was written so that it could be tested, and Chapter 4 reports the result for each.

Table 3.2: Non-functional requirements

ID	Requirement	How it is verified
N1	Portability: the whole system starts with one command, and Docker is the only prerequisite	Clean run from a fresh copy
N2	Scalability: the API scales horizontally without downtime or gateway changes	Throughput with 1 to 6 replicas
N3	Availability: the failure of one replica causes no visible outage	Failures counted under load
N4	Zero-downtime release: a new version rolls out with no failed requests	Rolling update under load
N5	Durability: data survives the deletion of its container	Delete and recreate the database
N6	Security: non-root, read-only containers, no capabilities, isolated networks	Runtime inspection
N7	Resilience: an outage of the cache degrades the service instead of stopping it	Redis stopped during use
N8	Observability: health and metrics are visible for every replica	Prometheus and Grafana

3.2 System design

3.2.1 System architecture

Figure 3.1 shows the design. A browser reaches the system only through the gateway, the single published port. The gateway serves as a reverse proxy: it sends page requests to the web container and API requests to one of the API replicas. API replicas keep no state in memory. Durable data lives in PostgreSQL, while Redis holds three kinds of short-lived data: the product cache, the job queue, and messages between replicas. Worker containers process orders in the background, so that a customer never waits for payment and packing to finish.

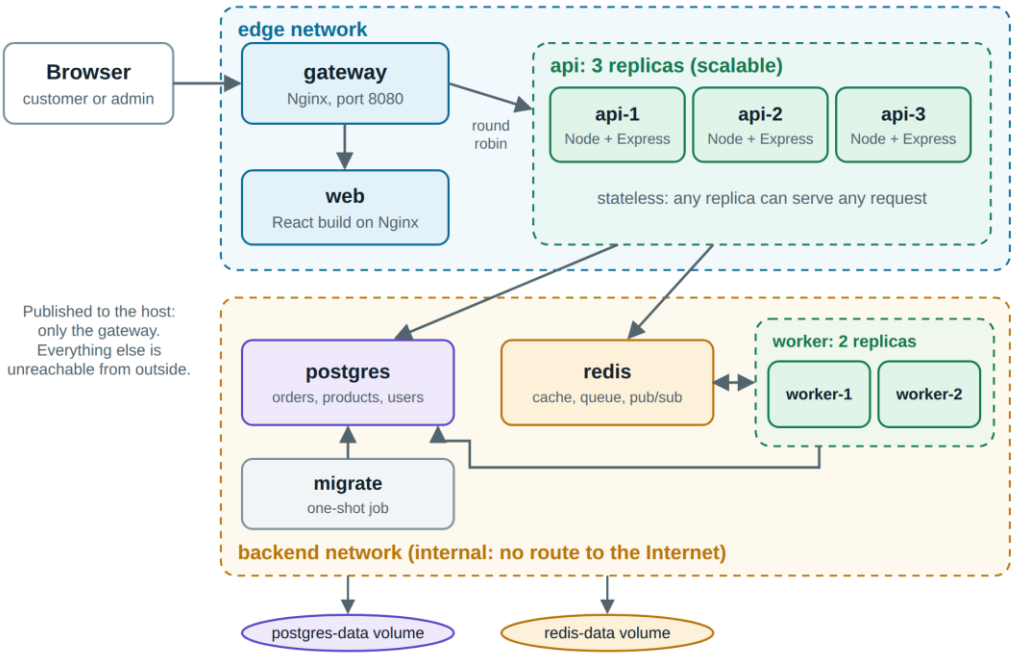


Figure 3.1: DockMart system architecture on Docker Compose

The API and the worker come from one image and differ only in the command they run. A third command, the migrator, applies the database schema and exits. Keeping one image for the three roles guarantees that they always share a version of the code. Our design separates the services into networks, so that the gateway and the web container have no route to the databases. A compromised front-end container cannot even attempt to connect to them, and Section 4.6 verifies this boundary by measurement.

3.2.2 Data design

The relational model in Figure 3.2 is deliberately small. Prices are stored as integer cents to avoid rounding errors, and each order item keeps its own copy of the unit price, so that later price changes do not alter past orders. The column processed_by records which worker last handled an order, which lets the website show the work moving between containers.

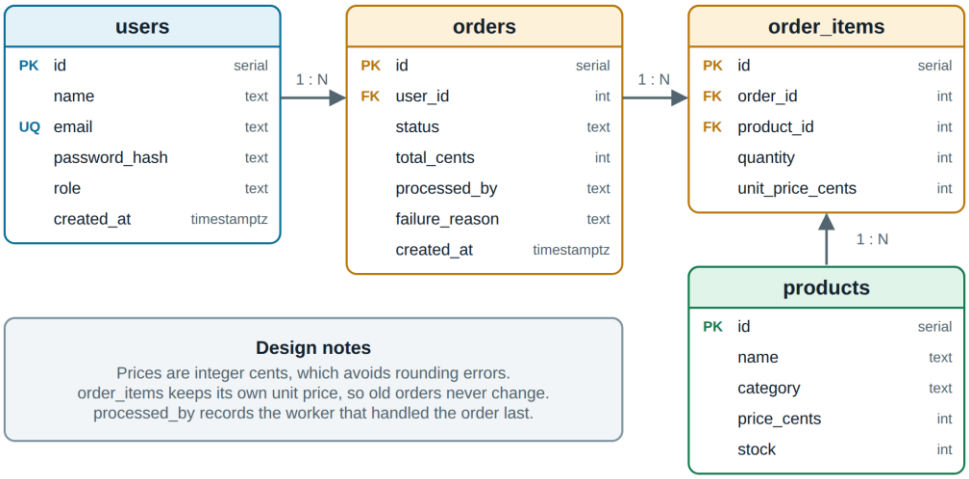


Figure 3.2: Entity-relationship diagram of the DockMart database

3.2.3 Order processing flow

Checkout is the most interesting flow, because it crosses every container. Figure 3.3 follows one order. The API locks the product rows, checks and reduces stock, and commits the order in one transaction. It then places a job in the queue and answers at once. A worker picks up the job and moves the order through its stages. Each change is published through Redis, which matters because the customer's browser is connected to one API replica and the worker is unrelated to it. Publishing to all replicas ensures that whichever one holds the connection can deliver the update.

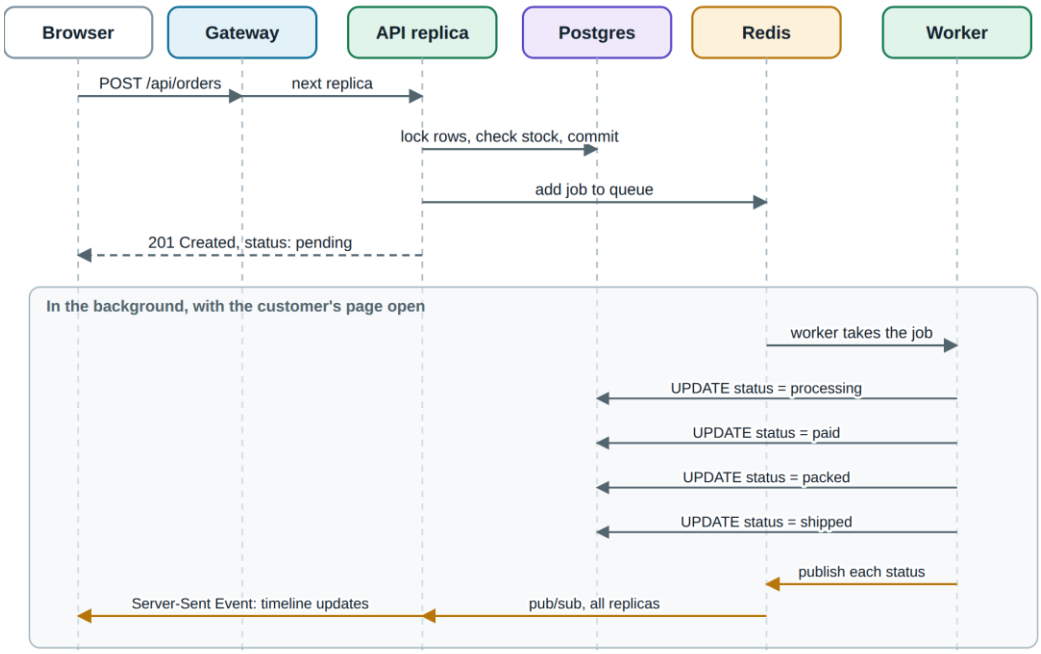


Figure 3.3: Sequence of an order, from checkout to the live status update

3.2.4 Technology choices

Table 3.3 lists the stack. We chose Node.js and React because the team knows them, which let us concentrate on the container layer. Versions of base images are pinned exactly, so that a build next year uses the same software as a build today.

Table 3.3: Technology stack

Layer	Technology	Role in the system
Front end	React 19, Vite 8	Single-page storefront and Ops Center
Back end	Node.js 24, Express 5	REST API with stateless replicas
Background jobs	BullMQ 6 on Redis	Reliable order queue, with retries
Database	PostgreSQL 18	Durable relational storage
Cache and messages	Redis 8	Product cache, queue, pub/sub, replica registry
Gateway	Nginx 1.31 (unprivileged image)	Reverse proxy, load balancer, rate limiting
Containers	Docker 29, Compose 5, Kubernetes 1.36	Images and orchestration

Layer	Technology	Role in the system
Quality	Prometheus 3, Grafana 13, k6, Grype, Vitest	Metrics, load and vulnerability tests

3.3 Implementation

3.3.1 Image design

Each service has its own Dockerfile. Figure 3.4 shows the multi-stage design of the backend image. A shared base stage copies only the package files, so that the dependency layers stay cached while the source code changes. From there, one branch installs all dependencies for development and testing, and another installs production dependencies only. The final stage starts again from a clean base image and copies in just what is needed to run.

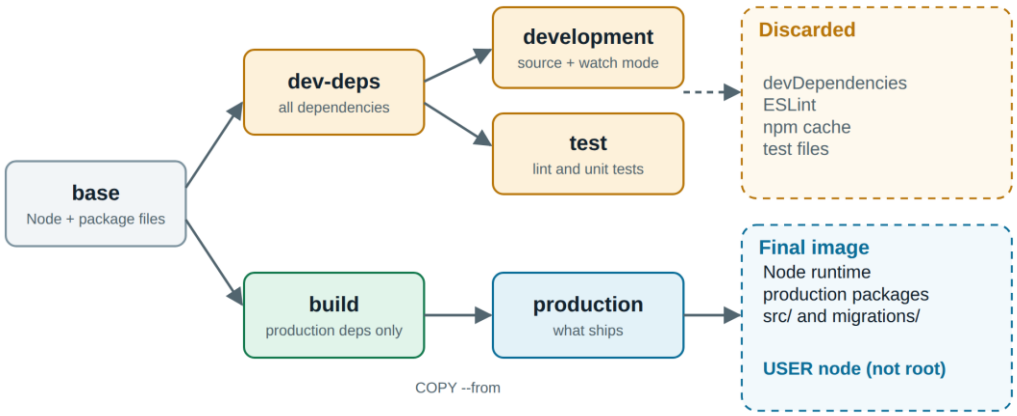


Figure 3.4: Multi-stage build of the backend image. Only the production stage ships

The test stage runs the linter and unit tests during the build, so an image that fails its tests cannot be produced at all. The final stage switches to the unprivileged node user and uses the exec form of CMD, so that Node.js receives stop signals directly. The front-end image takes this idea further: Node.js is needed only to compile the React application, and the final image is an unprivileged Nginx serving static files. Listing 3.1 shows the end of the backend Dockerfile.

Listing 3.1: Final stage of the backend Dockerfile

```
FROM node:${NODE_VERSION} AS production
WORKDIR /app
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --chown=node:node package.json ./
COPY --chown=node:node src ./src
COPY --chown=node:node migrations ./migrations
USER node
EXPOSE 4000
HEALTHCHECK --interval=10s --timeout=3s --retries=3 \
  CMD wget -q -O /dev/null http://127.0.0.1:4000/health/live || exit 1
CMD ["node", "src/server.js"]
```

3.3.2 The Compose stack

The file compose.yaml describes the whole system. Its most valuable feature is that start-up order depends on real readiness, and not merely on the order of launch. PostgreSQL and Redis must pass their health checks before anything else starts. The migrator must then exit successfully before the API and workers begin, and the gateway waits for healthy API replicas. Figure 3.5 shows this chain. Without it, the application would start before its database could accept connections, and the first requests would fail.

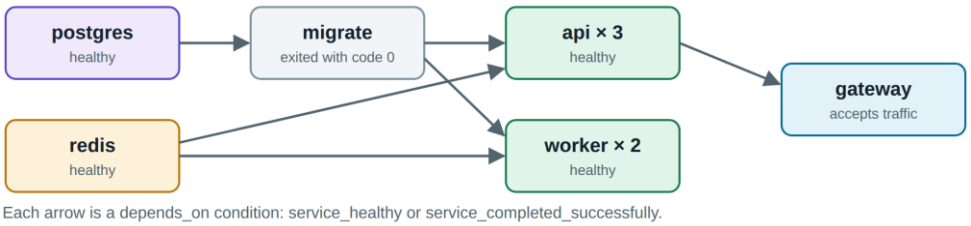


Figure 3.5: Start-up order, driven by health checks instead of fixed delays

Settings shared by several services, such as logging limits and security options, are defined once with YAML anchors. Passwords are stored as Compose secrets, which appear in the container as files under /run/secrets. The application reads a file when a variable such as DB_PASSWORD_FILE is present. This is the same convention that the official PostgreSQL image uses, and it means that the password never appears in docker inspect. In production mode the application refuses to start if a secret is missing, because failing at start-up is safer than running with a default password.

3.3.3 Dynamic load balancing

A gateway is only useful if it notices when replicas appear and disappear. Traditionally, Nginx resolves a server name once at start-up, so added replicas would receive no traffic until a reload. Since version 1.27.3, the open-source edition supports the resolve parameter, which makes Nginx re-read DNS while running and update its list of servers (NGINX, n.d.). Docker's DNS server returns one address per replica, so scaling needs no configuration change at all. We set the DNS validity to five seconds.

The gateway also retries a failed GET request on another replica, which hides most crashes from visitors. Nginx does not retry POST requests by default, and we rely on that, because repeating a checkout could create two orders. The configuration file is a template rendered at start-up. This lets the same gateway image use short service names under Compose and full names under Kubernetes, where the Nginx resolver does not apply the search domains from resolv.conf.

One setting deserves a note. Since version 1.29.7, Nginx keeps upstream connections open by default (NGINX, n.d.), but a Kubernetes Service balances traffic per connection and not per request. A kept connection would therefore tie each Nginx worker to one Pod. The gateway closes each upstream connection after use, which spreads requests evenly across the Pods at the small cost of opening a new connection every time.

3.3.4 Stateless replicas

Horizontal scaling requires that no replica holds anything that another replica needs. We reviewed each kind of state in the application and moved it out of the process, as Table 3.4 shows.

Table 3.4: Where each kind of state lives, so that replicas stay interchangeable

State	Common approach	DockMart
User session	In server memory	Signed JWT in an HttpOnly cookie, which any replica can verify
Cached products	A variable in the process	Redis, shared by all replicas
Live browser connections	Tied to one process	Redis pub/sub reaches the replica that holds the connection
Background jobs	A timer in the process	BullMQ queue in Redis
Business data	Local file	PostgreSQL on a named volume

The Ops Center needs to know which replicas exist. The usual way would be to mount the Docker socket, but that gives a container full control of the host. We use the design in Figure 3.6 instead. Every replica writes a short status record to Redis every two seconds, with an expiry of six seconds. A replica that dies stops renewing its record, which then disappears by itself. This needs no special privileges, and the same page works unchanged on Kubernetes, where no Docker socket exists.

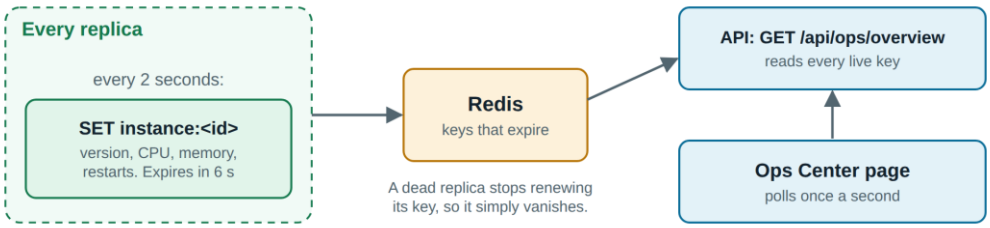


Figure 3.6: How the Ops Center discovers replicas without access to the Docker socket

3.3.5 Reliability

Three mechanisms keep the system available during failures. The first is graceful shutdown. On SIGTERM, a replica marks itself as not ready, closes live connections so that browsers reconnect elsewhere, finishes requests in progress and only then exits. The worker likewise waits for its current job.

The second is job recovery. A worker holds a short lock on each job. If it dies, the lock expires, and another worker takes the job over. To make this safe, processing resumes from the status stored in the database, so finished stages are not repeated and a customer cannot be charged twice.

The third is degraded operation when a dependency fails. Redis is an optimisation for reading the catalog, so an outage should not take the store down. Our first version failed this test: requests waited for Redis indefinitely, and the readiness check included Redis, so that one cache failure would have removed every replica from service at once. In the final version, cache errors fall through to PostgreSQL, the readiness check depends only on what a replica cannot work without, and checkout returns a clear 503 response before touching the stock. Section 4.6 verifies this behaviour.

3.3.6 Security hardening

Table 3.5 maps the security practices from Section 2.9 to their implementation. The hardened settings are defined once in compose.yaml and apply to every application container. PostgreSQL needs a few capabilities at start-up to prepare its data directory, and it receives only those.

Table 3.5: Security measures applied to the application containers

Threat	Measure	Implementation
Code runs with too much privilege	Non-root user	USER node in the image, user redis in Compose
Attacker modifies the container	Read-only filesystem	read_only: true, with tmpfs on /tmp only
Privilege escalation	No capabilities	cap_drop: ALL and no-new-privileges
Password leaks through inspection	Secrets as files	Compose secrets and Kubernetes Secrets
Lateral movement to the databases	Network segmentation	Internal network, NetworkPolicy

Threat	Measure	Implementation
One container exhausts the host	Resource limits	CPU and memory limits on each service
Brute-force sign-in	Rate limiting	Nginx limit_req on the login route
Known vulnerabilities in the image	Smaller images, scanning	Multi-stage builds and Grype

3.3.7 Monitoring

Containers appear and disappear, so a monitoring system cannot rely on a fixed list of hosts. Prometheus discovers replicas through the same DNS records that the gateway uses, and collects metrics from each one every five seconds. Grafana loads a prepared dashboard automatically. Monitoring is defined as an optional Compose profile, so it starts only when requested and does not burden an ordinary run. The metrics endpoints are never exposed through the gateway.

3.3.8 Kubernetes deployment

The manifests in the k8s folder describe the same system for Kubernetes. Table 3.6 shows how the Compose concepts translate. No application code changed, and the images are identical. Only the description of how to run them differs.

Table 3.6: How Compose concepts map to Kubernetes objects

Compose	Kubernetes	What improves
Service with replicas	Deployment and Service	Rolling updates and rollback
Named volume	StatefulSet and PersistentVolumeClaim	Stable identity and storage
Secret file	Secret, mounted as files	Same application code path
Internal network	NetworkPolicy	Rules per workload, not per network
Health check	Liveness and readiness probes	Unhealthy containers are acted upon
One-shot service	Job	Automatic retries

Compose	Kubernetes	What improves
--scale, by hand	HorizontalPodAutoscaler	Scaling on measured CPU use

A few additions make updates safe. The rolling update allows one extra Pod and no unavailable Pods. A preStop delay of five seconds keeps a terminating Pod serving until the removal of its endpoint has reached every node. A PodDisruptionBudget keeps at least two API replicas running during maintenance, and a topology spread constraint places replicas on different nodes. Pods run with the same hardening as under Compose, and NetworkPolicy objects recreate the separation between networks (Kubernetes, n.d.-e). We also turned off the automatic mounting of the service account token, because these Pods never call the Kubernetes API. We did not add an Ingress controller: the popular Ingress NGINX project reached its end of life in March 2026 (Kubernetes, 2025), and our own gateway image already does the job.

3.3.9 Development workflow and delivery

Containers should help during development as well as in production. The file compose.dev.yaml overrides the production settings for local work: it selects the development targets of the images, and runs one replica of each service. With Compose Watch, changes to source files are copied into the running containers (Docker Inc., n.d.-i), so the API restarts and the React page updates in a moment. New team members need nothing except Docker, because Node.js, PostgreSQL and Redis all run in containers.

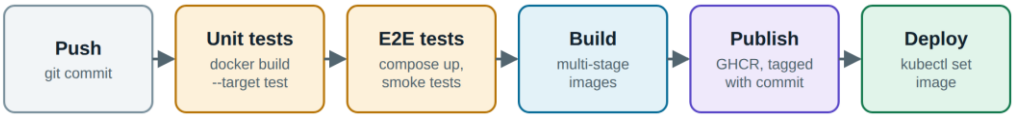


Figure 3.7: Delivery pipeline defined in the GitHub Actions workflow

The repository also contains a GitHub Actions pipeline, shown in Figure 3.7. We have not executed the pipeline itself, but each step in it was run by hand during the project.

3.4 Website walkthrough

The store opens on the page in Figure 3.8. Customers can search, filter by category and sort, and stock levels update live when someone else buys a product. A small note above the products shows whether the latest list came from the Redis cache, so that the effect of caching is visible while using the site. The footer of every page shows which replica answered the last request.

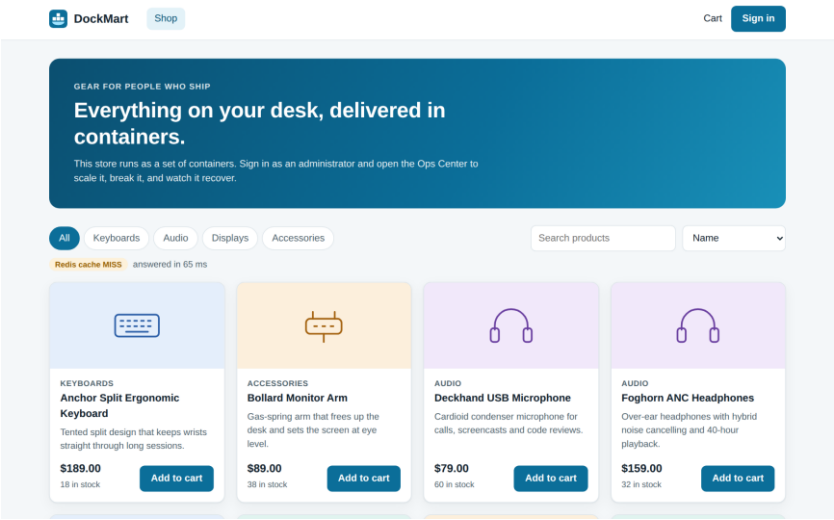


Figure 3.8: Storefront, with the cache status and the replica that served the page

Figure 3.9 shows an order in progress. The customer did not reload this page: the timeline moves forward as the worker publishes each stage, and the last line names the worker container.

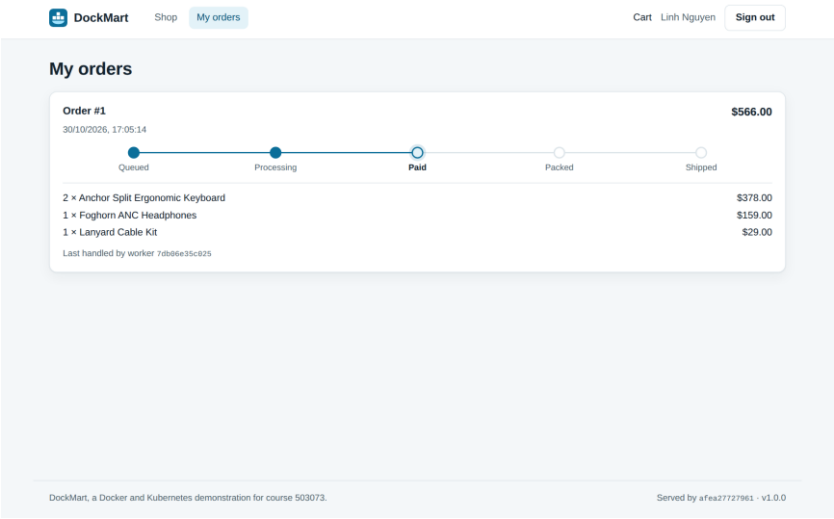


Figure 3.9: Live order tracking, updated through Server-Sent Events

The administrator manages products and sees all orders on a separate page. The Ops Center in Figure 3.10 is the centre of the demonstration. It lists every replica with its version, uptime, CPU, memory, restart count and work done. The load balancing panel sends 300 requests through the gateway and shows how many each replica answered. Buttons on each card crash that replica or make it fail its liveness probe, and the event log records what happened.

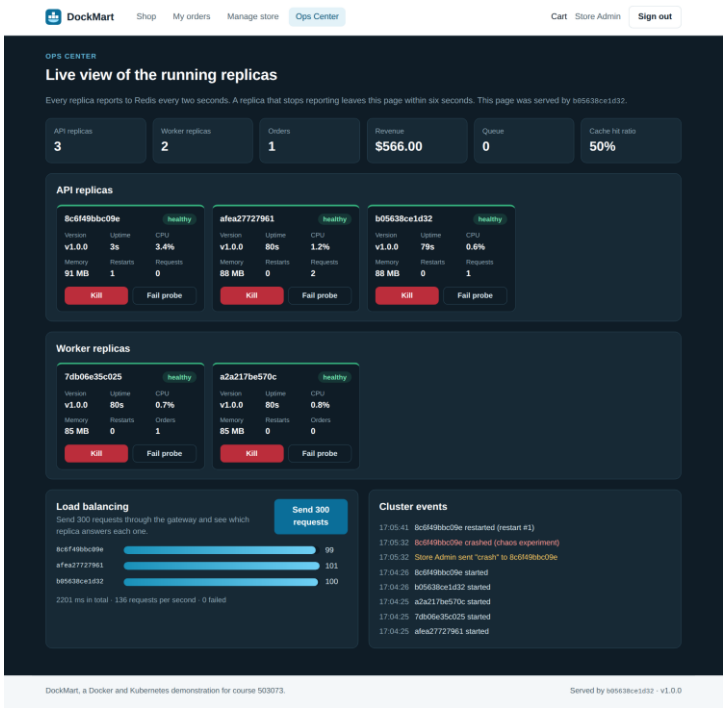


Figure 3.10: Ops Center just after one replica crashed: its uptime was reset, its restart count rose, and the event log recorded the crash

Figure 3.11 shows the Grafana dashboard from the optional monitoring profile, with request rate, CPU and memory for each replica.



Figure 3.11: Grafana dashboard, with replicas discovered automatically by Prometheus

CHAPTER 4. RESULT

4.1 Test environment and method

All experiments ran on one computer, described in Table 4.1. The load generator shares that machine with the system under test, so absolute throughput says little about production hardware. Our interest is in the comparisons: between build strategies, between replica counts, and between a healthy and a failing system. Build times are the median of three runs, and each scaling point is the median of three runs of 20 seconds. The scripts and raw data are included in the source code, in the scripts and results folders.

Table 4.1: Test environment

Item	Value
Host	Windows 11 Pro, 12 logical processors, Docker Desktop on WSL 2
Memory available to Docker	7.4 GiB
Docker and Compose	Docker Engine 29.8.2, Compose 5.5.1
Kubernetes	Version 1.36.1, three nodes provisioned by Docker Desktop with kind
Load generator	k6 2.3.0, running in a container on the same machine
Vulnerability scanner	Grype 0.121.0

4.2 Functional testing

Table 4.2 summarises the automated tests. The end-to-end suite runs against the real stack from inside a container, so it needs no tools on the host. Beyond ordinary flows, it checks properties that matter more in a distributed system: a session is accepted by every replica, six customers racing for the last two units never oversell the stock, each stage of an order reaches the browser, and internal endpoints such as metrics are not exposed by the gateway. The same suite passed unchanged on both platforms, which is direct evidence of portability.

Table 4.2: Automated test results

Level	Tool	Where it runs	Passed
Backend unit tests and lint	Node test runner, ESLint	Docker build, test target	17 of 17
Frontend unit tests	Vitest	Docker build, test target	6 of 6
End-to-end tests	Node test runner	Compose stack	24 of 24
End-to-end tests	Node test runner	Kubernetes cluster	24 of 24

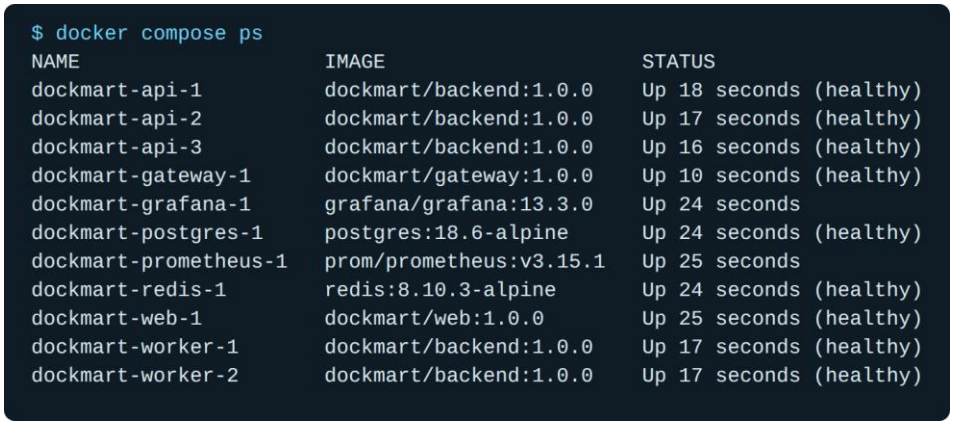


Figure 4.1: All services running and healthy on Docker Compose (real output)

4.3 Image build strategies

We built the same backend in four ways: a naive Dockerfile of the kind found in many tutorials, a Debian slim image, a distroless image and our multi-stage Alpine image. Table 4.3 and Figure 4.2 show the results. The multi-stage image uses 6.0 times less disk space than the naive one, and it downloads in 6.2 times fewer bytes. The naive image carries the full Debian toolchain, all development dependencies and the npm cache, and none of these are needed at run time. Disk use is the figure that Docker reports. With the containerd image store it is larger than the download size, which counts only the compressed layers.

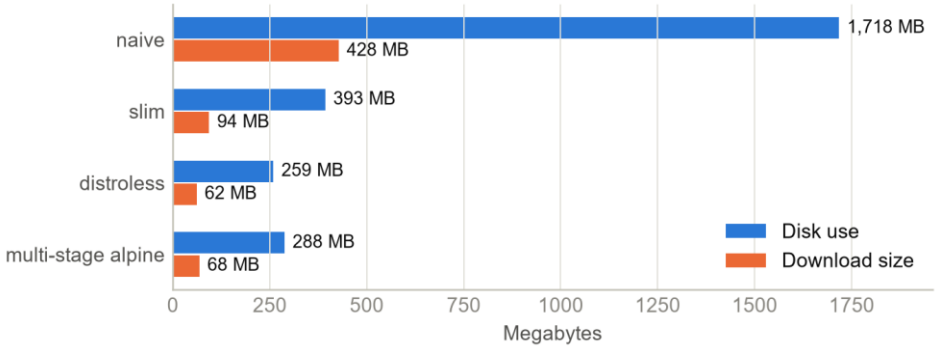


Figure 4.2: Size of the same application under four build strategies

Table 4.3: Comparison of image build strategies for the same backend

Metric	Naive	Slim	Distroless	Multi-stage Alpine
Disk use (MB)	1,718	393	259	288
Download size (MB)	428	94	62	68
Layers	11	10	27	9
First build (s)	16.5	12.8	26.3	13.6
Rebuild after code change (s)	4.1	3.6	1.7	2.9
Runs as	root	node	non-root	node
Shell included	Yes	Yes	No	Yes
Critical vulnerabilities	57	2	0	0
High vulnerabilities	373	76	11	6
Medium vulnerabilities	727	82	10	13

Size also affects security. The scanner reported 57 critical and 373 high vulnerabilities in the naive image, against 0 and 6 in ours, because a package that is not in the image cannot be vulnerable. These counts are a snapshot of the vulnerability database on the day of the scan, and they will change as new issues are published. Distroless was slightly smaller than our image, and it has no shell, so an attacker who gets in has fewer tools. Its first build took the longest, however, it is harder to debug, and our health check would need rewriting because the image has no wget. As Alpine came out with no critical findings and the fewest high ones, we

kept it. Distroless remains a reasonable choice for a team that wants the smallest possible attack surface.

Figure 4.3 shows build times, and here our expectations were only partly met. We expected the naive build to be the fastest, because it has the fewest steps, but ours took 13.6 s against 16.5 s, since it installs fewer packages. After a one-line code change, the naive image needed 4.1 s, because copying the source before installing dependencies invalidates the npm layer, while ours needed 2.9 s. That saving is real but modest, because DockMart has few dependencies and reinstalling them is quick. A project with hundreds of packages would gain far more. Build times also depend on what else the machine is doing, so these figures are best read as a comparison between strategies.

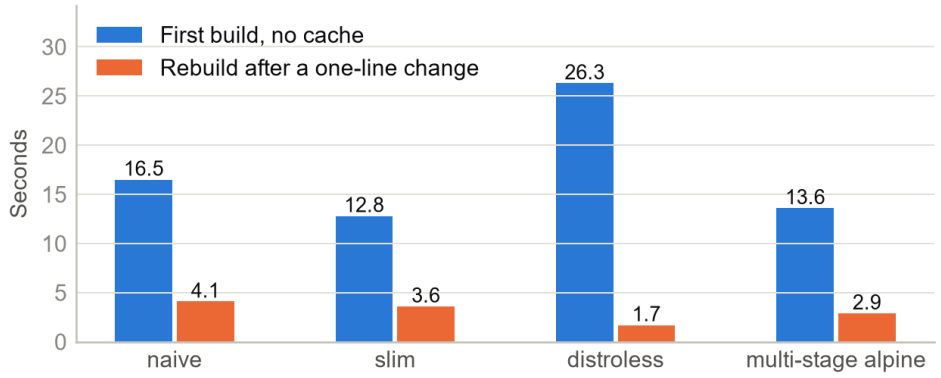


Figure 4.3: Build time, first build and rebuild after a one-line change

4.4 Start-up time and resource usage

Running a minimal container and removing it took a median of 0.66 s in total. Starting the complete stack from nothing, until all services were healthy, took 26.1 s, and most of that time goes to the health-check intervals of the dependency chain. Adding three API replicas to a running system took 9.0 s. Figure 4.4 shows memory use at idle. The nine running containers used about 227 MB in total. Each Nginx container needs only a few megabytes, which shows the difference from a virtual machine, where each instance would first need memory for a guest operating system.

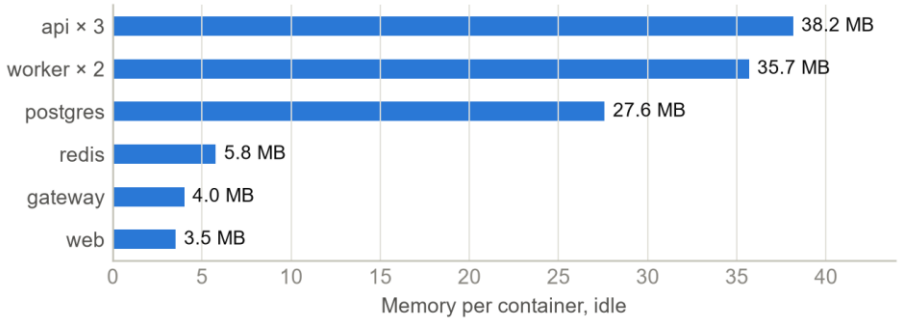


Figure 4.4: Memory used by each container at idle

4.5 Horizontal scaling

We sent CPU-intensive requests through the gateway, with 48 virtual users, and increased the number of API replicas from one to six. Each replica is limited to half a CPU core, so a higher count really adds capacity. Figure 4.5 and Table 4.4 show the outcome. Throughput rose from 107 to 510 requests per second, a factor of 4.8, and no request failed at any level. The gateway was never restarted. Latency improved overall, from 507 ms to 284 ms at the 95th percentile, but it did not fall smoothly: it stayed between 413 and 485 ms from two to five replicas, so we read the individual latency points with caution and rely on the throughput trend.

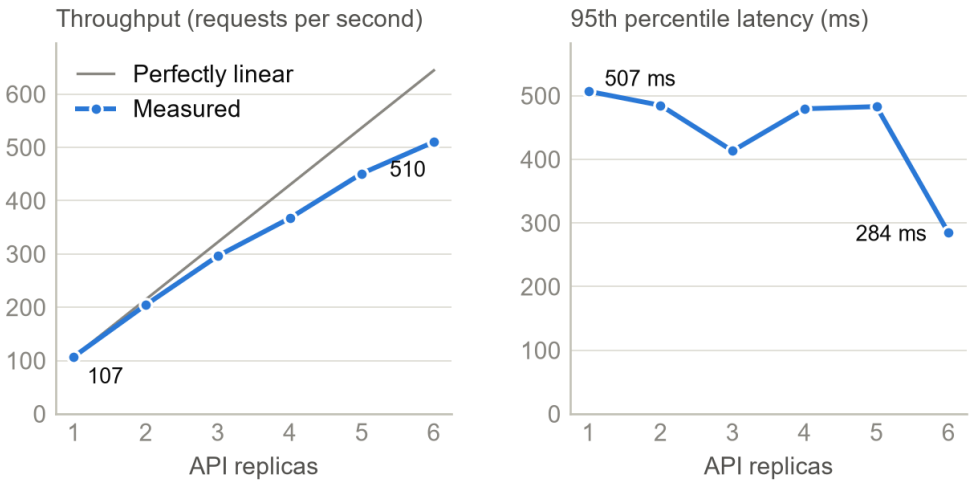


Figure 4.5: Throughput and latency as replicas are added

Table 4.4: Throughput and latency by number of API replicas

Replicas	Requests per second	p95 latency (ms)	Speed-up	Failed requests
1	107.4	507	1.00×	0.00%
2	204.6	485	1.91×	0.00%
3	296.3	413	2.76×	0.00%
4	367.2	480	3.42×	0.00%
5	451.1	483	4.20×	0.00%
6	510.2	284	4.75×	0.00%

The gap from perfectly linear scaling has ordinary causes: the load generator shares the same machine, the gateway has its own limit of half a CPU core, and load balancing has a small cost. Scaling works here because this particular work is CPU-bound and independent. A workload limited by the single PostgreSQL instance would stop scaling much earlier, however many API replicas we added.

4.6 Failure behaviour on Docker Compose

We kept a steady load of 100 requests per second for 40 seconds while causing failures on purpose. Table 4.5 shows the results. When an API process crashed, the gateway moved its traffic to the other replicas, and Docker restarted the container. A graceful stop behaved as designed, because the replica drains before it exits.

Table 4.5: Failure experiments on Docker Compose

Scenario	Failed requests	What happened
Baseline, no failure	0 of 4,001	Reference run
API process crashes	0 of 4,001	Restart policy recovered the replica in 6.6 s
docker stop (SIGTERM)	0 of 4,001	Graceful shutdown took 0.7 s
docker kill (SIGKILL)	0 of 4,001	Container stayed exited, as documented

A hard docker kill gave a different result from a crash in two ways. The container stayed stopped, because Docker treats this as an operator's decision (Docker Inc., n.d.-d). In addition, the container's address disappears from DNS immediately, and in an earlier run, with one request sent at a time, this produced one 502 response out of 300. The gateway log showed a timeout on the dead address, followed by no live upstreams. The Nginx source code explains this: when a DNS refresh replaces the server list during a retry, the retry stops. The window is narrow, and a real process crash does not trigger it, because the address stays registered. We report this as a known limit of the approach, and not as a solved problem.

Background work also survived failure. We crashed the worker that was processing an order. Its lock on the job expired, the job became available again, and the order reached shipped 9.8 s after the crash. In this run the restarted container itself picked the job up, although the other worker could equally have done so. Either way, the worker resumes from the status stored in the database, so finished stages are not repeated. In the Redis outage test, the catalog still answered with status 200 in 28 ms, marked BYPASS, sessions kept working, and checkout returned 503 with a clear message. After Redis returned, orders succeeded again (status 201). Deleting the PostgreSQL container and creating a new one lost no data.

Table 4.6 shows the measured connectivity between services, tested from inside each container. It confirms the design from Chapter 3: the gateway and the web container cannot reach the databases, and the worker and data services have no route to the Internet. The worker can reach the API, but only through the monitoring network, which Prometheus needs. Inside the API container, the process runs as an unprivileged user, the filesystem rejects writes and the effective capability set is empty.

Table 4.6: Measured connectivity between services

From \ To	web	api	postgres	redis	Internet
gateway	Yes	Yes	No	No	Yes
web	Yes	Yes	No	No	Yes
api	Yes	Yes	Yes	Yes	Yes
worker	No	Yes	Yes	Yes	No
postgres	No	Yes	Yes	Yes	No
redis	No	Yes	Yes	Yes	No

4.7 Kubernetes

4.7.1 Availability

Table 4.7 shows the experiments on the three-node cluster. Each ran for 75 seconds under 100 requests per second, from a k6 job on the control plane. No experiment lost a single request, although the slowest request reached about one second in three of them, so a few visitors would have noticed a short delay. Figure 4.6 shows why the rolling update is safe: as version 1.1.0 comes up, version 1.0.0 goes down at the same pace, and the number of ready Pods never drops below three. The update took 14.2 s and the rollback 15.2 s.

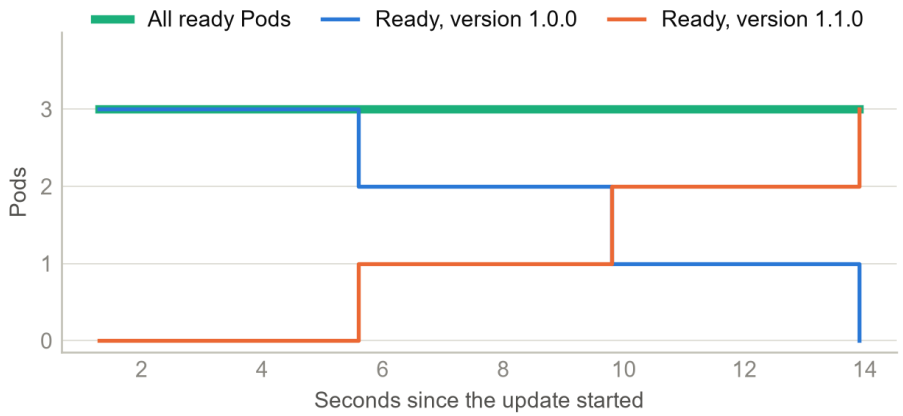


Figure 4.6: Ready Pods by version during a rolling update

Table 4.7: Availability experiments on Kubernetes

Experiment	Failed requests	Duration	Slowest request
Rolling update, 1.0.0 to 1.1.0	0 of 7,500	14.2 s	1018 ms
Rollback to 1.0.0	0 of 7,501	15.2 s	134 ms
Pod deleted	0 of 7,501	4.7 s to replace	81 ms
Liveness probe failure	0 of 7,501	21.2 s to recover	1034 ms
Node drained	0 of 7,501	16.7 s	1036 ms

```
$ kubectl -n dockmart get pods -o wide
NAME                       READY   STATUS    RESTARTS   NODE
api-744f46855b-459gp       1/1     Running   0          desktop-worker
api-744f46855b-bgd2d       1/1     Running   0          desktop-worker2
api-744f46855b-x99rr       1/1     Running   0          desktop-worker
gateway-869d689bdb-fcbr4   1/1     Running   0          desktop-worker2
gateway-869d689bdb-rbcwz   1/1     Running   0          desktop-worker
postgres-0                 1/1     Running   0          desktop-worker
redis-0                    1/1     Running   0          desktop-worker
web-6b4bd679f-9x8lb        1/1     Running   0          desktop-worker
web-6b4bd679f-grgqz        1/1     Running   0          desktop-worker2
worker-595f5445c8-8j96f    1/1     Running   0          desktop-worker
worker-595f5445c8-p7v6k    1/1     Running   0          desktop-worker2
```

Figure 4.7: DockMart Pods spread across the two worker nodes of the cluster (real output)

Two results show what an orchestrator adds to Compose. When a replica failed its liveness probe, Kubernetes restarted it after 17.8 s without any human action, whereas Compose would only have labelled it unhealthy. Draining a node, the normal first step of server maintenance, moved its workloads elsewhere in 16.7 s, with the PodDisruptionBudget allowing only one API replica to be evicted at a time. One thing we noticed is that Kubernetes does not rebalance after the node returns: all three API replicas stayed on the same node until the next rollout.

4.7.2 Autoscaling

To test the HorizontalPodAutoscaler, we applied CPU-intensive load for two minutes. Figure 4.8 shows the result. CPU use rose far above the 70% target, and the autoscaler added replicas until it reached its maximum of 8, about 60 seconds after the load began. Average CPU then fell as the load spread out. The system

served 309 requests per second with no failures. After the load stopped, the autoscaler returned to three replicas without any intervention. The delay at each end is expected: metrics are collected every 15 seconds, and we set a stabilisation window of 30 seconds to prevent replicas from being removed too eagerly.

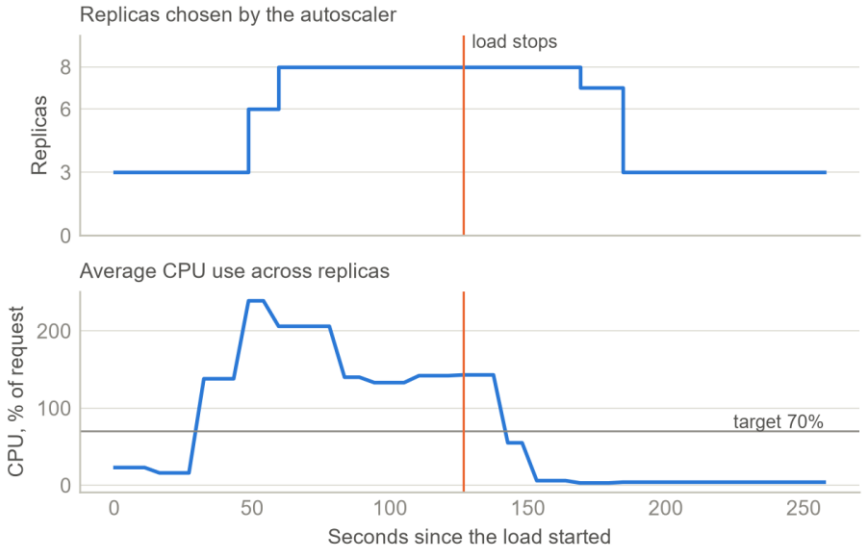


Figure 4.8: Autoscaling under load: replicas and average CPU use

4.7.3 Network policies

We checked the NetworkPolicy objects from inside the Pods. The gateway could reach the web Service but timed out on PostgreSQL and Redis, while the API reached both. Kubernetes therefore enforces the same boundaries as the Compose networks, through a different mechanism.

4.8 Problems encountered

Much of what we learned came from things that did not work the first time. Table 4.8 lists the main problems. Several of them only appeared because we tested failure deliberately. A demonstration that only follows the happy path would have hidden all of them.

Table 4.8: Problems met during the project, and how we solved them

Problem	Cause	Solution
Redis would not start after we dropped all capabilities	Starting through sh -c skipped the image's step that switches to the redis user, so it ran as root without permission on /data	Run the container as user redis, which needs no capabilities at all
A killed replica did not come back	Docker treats docker kill as an operator's decision, so the restart policy does not apply	Documented it, and made the Ops Center crash the process from inside
One request in 300 returned 502 during a hard kill	Nginx stops retrying when a DNS refresh replaces the server list while a retry is in progress	Accepted: the window is very small. The source code confirmed the cause
The website hung when Redis stopped	The Redis client queued commands for ever, and readiness depended on Redis	Fail fast, fall back to PostgreSQL, and leave Redis out of readiness
The catalog still took 8.5 s during a Redis outage	Each cache command waited for one reconnect attempt before it failed	Skip Redis at once while the connection is down, which brought the response to 28 ms
Pods failed to start on Kubernetes	A read-only Secret on /run/secrets blocked the service account token, because /var/run links to /run	Set automountServiceAccount Token to false, which is also safer
A liveness failure lost two requests	The failing Pod kept receiving traffic until its restart	Make readiness fail as well, so traffic leaves the Pod first
Load reached only two of the three Pods, and sometimes one	Nginx 1.29.7 and later keep upstream connections open, and a Service balances per connection, so each Nginx worker stayed on one Pod	Close the upstream connection after each request, which gave an even split across all Pods
Workers could reach the Internet	The monitoring network was not internal	Made it internal, and added a separate network for published ports
The gateway could not resolve api on Kubernetes	The Nginx resolver ignores the search list in resolv.conf	Pass full service names through template variables

4.9 Discussion

Table 4.9 gives the status of each non-functional requirement. All were met, with one qualification: zero-downtime release needs Kubernetes, because Compose recreates containers. Table 4.10 compares the two platforms directly.

Table 4.9: Non-functional requirements and results

ID	Requirement	Result	Status
N1	Portability	One command starts the stack in 26.1 s, and the same images run on Kubernetes	Met
N2	Scalability	4.8× throughput with six replicas, with no gateway change	Met
N3	Availability	0 failed requests during a process crash	Met
N4	Zero-downtime release	0 failed of 7,500 on Kubernetes. Compose has no rolling update	Met on Kubernetes
N5	Durability	Orders survive deleting the database container	Met
N6	Security	Non-root, read-only, no capabilities, isolated networks	Met
N7	Resilience	Catalog served in 28 ms during a Redis outage, and checkout returns 503	Met
N8	Observability	Replicas discovered automatically, with a provisioned dashboard	Met

Table 4.10: What each platform delivered in our measurements

Capability	Docker Compose	Kubernetes
Commands to deploy	One	Several, plus a cluster to run
Scaling	Manual, one command	Automatic, 3 to 8 replicas
Process crash	Restarted, 0 failed requests	Restarted, and the Pod is replaced if needed
Unhealthy but running	Reported only	Restarted after 17.8 s, 0 failed requests
New version	Containers are recreated	Rolling, 0 failed requests
Rollback	Redeploy the old tag	One command, 15.2 s
Machine failure	Total outage	Workload moves to other nodes

The most important finding is that containers enable these results but do not produce them on their own. Replicas could be added because we made the API stateless. Crashes were invisible because the gateway retries and the worker resumes from stored state. Rolling updates lost no requests because the application handles SIGTERM and the Pods delay their shutdown. In each case the platform provided a mechanism, and the application had to cooperate with it.

These results have limits. The three Kubernetes nodes are containers on one computer, so we could simulate maintenance but not the real loss of a machine. All load came from that same computer, which limits the absolute numbers. PostgreSQL and Redis run as single instances, so they remain single points of failure on both platforms, and no amount of API scaling changes that. Finally, each failure experiment ran only a few times, so a rare race like the 502 described in Section 4.6 could exist elsewhere without our having seen it.

CHAPTER 5. CONCLUSION

5.1 Conclusion

A container turned out to be a simple idea: an ordinary process that the kernel isolates with namespaces and limits with cgroups. Docker's contribution was a practical way to package that process, and the OCI standard made the package portable.

DockMart allowed us to check each claim with numbers. Portability held: the same images passed the same 24 end-to-end tests on Docker Compose and on Kubernetes. Consistency came from building development, test and production targets from one Dockerfile. Efficiency was clear in the 6.0-fold reduction in image size and the small memory footprint of the stack. Scalability gave 4.8 times the throughput with six replicas, and Kubernetes delivered updates, rollbacks and node maintenance without losing a request.

The comparison also showed that more tooling is not automatically better. Kubernetes earned its complexity only for what Compose cannot do: survive the loss of a machine, replace containers gradually, and scale by itself. Our main reflection is that the platform is only half of the story, as the results depended on statelessness, graceful shutdown, idempotent jobs and tolerance of a failed dependency. We also learned more from testing failures than from the parts that worked.

5.2 Potential and Development Direction

Several improvements follow directly from the limits in Section 4.9. PostgreSQL and Redis should become replicated or managed services, to remove the remaining single points of failure. The system should run on a real cloud cluster with HTTPS at the edge, so that the loss of a machine can be tested for real, and the gateway could move to the Gateway API, which the Kubernetes project now recommends in place of Ingress (Kubernetes, 2025). The packaging idea that

Docker popularised in 2013 looks likely to remain the basic unit of web deployment for a long time.

REFERENCES

Bernstein, D. (2014). Containers and cloud: From LXC to Docker to Kubernetes. IEEE Cloud Computing, 1(3), 81-84. https://doi.org/10.1109/MCC.2014.51

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. Communications of the ACM, 59(5), 50-57. https://doi.org/10.1145/2890784

Cloud Native Computing Foundation. (2026, January 20). Kubernetes established as the de facto "operating system" for AI as production use hits 82% in 2025 CNCF annual cloud native survey. https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/

Docker Inc. (n.d.-a). Docker overview. Docker Docs. https://docs.docker.com/get-started/docker-overview/

Docker Inc. (n.d.-b). Building best practices. Docker Docs. https://docs.docker.com/build/building/best-practices/

Docker Inc. (n.d.-c). Multi-stage builds. Docker Docs. https://docs.docker.com/build/building/multi-stage/

Docker Inc. (n.d.-d). Start containers automatically. Docker Docs. https://docs.docker.com/engine/containers/start-containers-automatically/

Docker Inc. (n.d.-e). Networking overview. Docker Docs. https://docs.docker.com/engine/network/

Docker Inc. (n.d.-f). Storage. Docker Docs. https://docs.docker.com/engine/storage/

Docker Inc. (n.d.-g). Docker Compose overview. Docker Docs. https://docs.docker.com/compose/

Docker Inc. (n.d.-h). OverlayFS storage driver. Docker Docs. https://docs.docker.com/engine/storage/drivers/overlayfs-driver/

Docker Inc. (n.d.-i). Use Compose Watch. Docker Docs. https://docs.docker.com/compose/how-tos/file-watch/

Docker Inc. (n.d.-j). Docker pricing FAQs. https://www.docker.com/pricing/faq/

Docker Inc. (n.d.-k). Swarm mode. Docker Docs. https://docs.docker.com/engine/swarm/

Felter, W., Ferreira, A., Rajamony, R., & Rubio, J. (2015). An updated performance comparison of virtual machines and Linux containers. 2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), 171-172. https://doi.org/10.1109/ISPASS.2015.7095802

HashiCorp. (n.d.). What is Nomad? Nomad Documentation. https://developer.hashicorp.com/nomad/docs/what-is-nomad

Hykes, S. (2013, March). The future of Linux containers [Conference presentation]. PyCon US 2013, Santa Clara, CA, United States. https://pyvideo.org/pycon-us-2013/the-future-of-linux-containers.html

Kamp, P.-H., & Watson, R. N. M. (2000). Jails: Confining the omnipotent root. Proceedings of the 2nd International SANE Conference.

Kerrisk, M. (n.d.-a). capabilities(7): Overview of Linux capabilities. Linux manual pages. https://man7.org/linux/man-pages/man7/capabilities.7.html

Kerrisk, M. (n.d.-b). namespaces(7): Overview of Linux namespaces. Linux manual pages. https://man7.org/linux/man-pages/man7/namespaces.7.html

Kerrisk, M. (n.d.-c). cgroups(7): Linux control groups. Linux manual pages. https://man7.org/linux/man-pages/man7/cgroups.7.html

Kubernetes. (2022, February 17). Updated: Dockershim removal FAQ. Kubernetes Blog. https://kubernetes.io/blog/2022/02/17/dockershim-faq/

Kubernetes. (2025, November 11). Ingress NGINX retirement: What you need to know. Kubernetes Blog. https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/

Kubernetes. (n.d.-a). Kubernetes components. Kubernetes Documentation. https://kubernetes.io/docs/concepts/overview/components/

Kubernetes. (n.d.-b). Deployments. Kubernetes Documentation. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

Kubernetes. (n.d.-c). Liveness, readiness, and startup probes. Kubernetes Documentation. https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/

Kubernetes. (n.d.-d). Horizontal Pod Autoscaling. Kubernetes Documentation. https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

Kubernetes. (n.d.-e). Network policies. Kubernetes Documentation. https://kubernetes.io/docs/concepts/services-networking/network-policies/

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239), Article 2.

National Institute of Standards and Technology. (2019). CVE-2019-5736 detail. National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2019-5736

National Institute of Standards and Technology. (2024). CVE-2024-21626 detail. National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2024-21626

NGINX. (n.d.). Module ngx_http_upstream_module. nginx documentation. https://nginx.org/en/docs/http/ngx_http_upstream_module.html

Open Container Initiative. (n.d.). About the Open Container Initiative. https://opencontainers.org/about/overview/

OWASP. (2024). Docker security cheat sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

Podman. (n.d.). What is Podman? Podman Documentation. https://docs.podman.io/en/latest/

Price, D., & Tucker, A. (2004). Solaris Zones: Operating system support for consolidating commercial workloads. Proceedings of the 18th Large Installation System Administration Conference (LISA '04), 241-254.

Soltesz, S., Pötzl, H., Fiuczynski, M. E., Bavier, A., & Peterson, L. (2007). Container-based operating system virtualization: A scalable, high-performance alternative to hypervisors. Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems, 275-287. https://doi.org/10.1145/1272996.1273025

Souppaya, M., Morello, J., & Scarfone, K. (2017). Application container security guide (NIST Special Publication 800-190). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-190

Stack Overflow. (2025). 2025 Stack Overflow Developer Survey: Technology. https://survey.stackoverflow.co/2025/technology

Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., & Wilkes, J. (2015). Large-scale cluster management at Google with Borg. Proceedings of the Tenth European Conference on Computer Systems (EuroSys '15), Article 18. https://doi.org/10.1145/2741948.2741964

Wiggins, A. (2017). The twelve-factor app. https://12factor.net/

APPENDIX A. RUNNING THE PROJECT

The README file in the source folder contains the full, tested instructions. In short, Docker is the only requirement. From the source folder, run docker compose up -d --build --wait and open http://localhost:8080. The end-to-end tests run with docker compose --profile test run --rm smoke.

Role	Email or user	Password
Administrator	admin@dockmart.local	DockMart-Admin-2026
Customer	Register on the sign-in page	Chosen by the user
Grafana	admin	DockMart-Admin-2026

The passwords above are demonstration values, stored in the secrets folder so that the project starts with one command.

APPENDIX B. REST API

Method and path	Access	Purpose
POST /api/auth/register	Public	Create a customer account
POST /api/auth/login	Public, rate limited	Sign in
POST /api/auth/logout	Public	Sign out
GET /api/auth/me	Signed in	Current user
GET /api/products	Public	List, search, filter and sort products
GET /api/products/:id	Public	One product
POST /api/products	Admin	Create a product
PUT /api/products/:id	Admin	Update a product
DELETE /api/products/:id	Admin	Delete a product
POST /api/orders	Signed in	Place an order
GET /api/orders	Signed in	Own orders, or all orders for an admin
GET /api/orders/:id	Owner or admin	One order
GET /api/events	Signed in	Server-Sent Events stream
GET /api/whoami	Public	The replica that answered
GET /api/ops/overview	Admin	Replicas, queue, cache, totals, events
POST /api/ops/chaos	Admin	Crash a replica, or fail or heal its probe
GET /api/ops/compute	Demo only	CPU work for load tests